

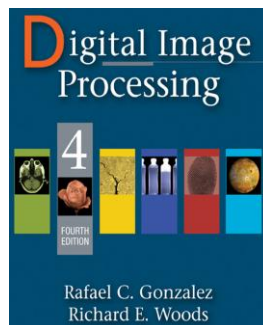


دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گزارش تمرین دوم – تصویرپردازی رقمی

مورفولوژی



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،

Aminkianiworkeng@gmail.com

استاد درس: دکتر نقش نیلچی

نیمسال دوم تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

مستندات	۳
۰- مسئله و تحلیل کلی آن:	۳
۱- تمرین اول: Dilation	۶
۲- تمرین دوم: Erosion	۱۰
۳- تمرین سوم: Opening	۱۳
۴- تمرین چهارم: Closing	۱۶
۵- تمرین پنجم: Hit-or-Miss Transform	۱۹
۶- مراجع	۲۲

مستندات

۰- مسئله و تحلیل کلی آن:

در این مجموعه تمرین، هدف این است که عملیات پایه‌ی مورفولوژی ریاضی را روی تصاویر دودویی و خاکستری ببینیم و بفهمیم هر عمل چه اثری روی ساختارهای روشن/تیره‌ی تصویر می‌گذارد. برای هر مثال، ترجمه‌ی مستقیم منطق MATLAB به Python انجام شده است تا همان رفتار مفهومی حفظ شود، اما کد برای اجرای راحت در Google Colab آماده شده است. ساختارهای مورفولوژیک اینجا فقط ابزار نمایش نیستند؛ آن‌ها برای اتصال قطعات شکسته، حذف اجسام کوچک، صاف کردن مرزها، و تشخیص الگوهای خاص استفاده می‌شوند.

منابع تصویری مورد استفاده

تصویر circles از یک مخزن عمومی GitHub گرفته شده است.

تصویر blobs از مخزن عمومی samples.fiji.sc دریافت شده است.

تصویر snowflakes از یک مخزن عمومی GitHub گرفته شده است.

تصویر cameraman از یک مجموعه‌ی استاندارد عمومی تصویر برای پردازش تصویر دریافت شده است.

در نوت‌بوک، برای reproducibility، همه‌ی این تصاویر از آدرس‌های عمومی قابل دانلود شده‌اند و نیازی به آپلود دستی فایل‌ها نیست. همچنین در Colab کافی است این بسته‌ها نصب شوند:

```
!pip -q install numpy matplotlib pillow scipy scikit-image ...
```

MATLAB	Python
zeros(256,256)	<code>np.zeros((256, 256), dtype=bool)</code>
B(i,j)=1	<code>B[i, j] = True</code>
ones(k)	<code>np.ones((k, k), dtype=bool)</code>
imdilate	<code>scipy.ndimage.binary_dilation</code>
imerode	<code>scipy.ndimage.binary_erosion</code>
im2bw(I,0.3)	<code>I > int(0.3*255)</code>
imshow(...), title(...)	<code>matplotlib.pyplot.imshow(..., cmap='gray')</code>

Dilation: $A \oplus B$

Erosion: $A \ominus B$

Closing: $A \bullet B = (A \oplus B) \ominus B$

Opening: $A \circ B = (A \ominus B) \oplus B$

Hit-and-Miss: $A \otimes (B1, B2) = (A \ominus B1) \cap (Ac \ominus B2)$

- ایده‌ی اصلی پردازش مورفولوژیک این است که به‌جای کار با شدت روشنایی پیکسل‌ها، با شکل و ساختار اشیا کار کنیم.

- در تصاویر باینری، هر پیکسل فقط دو حالت دارد:

• ۱ یا سفید: عضو شیء

• ۰ یا سیاه: پس‌زمینه

- عملیات مورفولوژیک با استفاده از یک عنصر کوچک به نام **Structuring Element** یا عنصر سازنده انجام می‌شوند. این عنصر مثل یک «الگو» یا «قالب» است که روی تصویر حرکت می‌کند و مشخص می‌کند چه اتفاقی روی شکل‌ها بیفتد. که در این تمرین:

`strel('disk', r)` در MATLAB یعنی یک عنصر سازنده‌ی دیسکی (دایره‌ای) با شعاع `r`.

چرا دیسک؟

چون وقتی شکل‌ها یا اشیای تصویر حالت گرد، تقریباً گرد، یا بدون جهت خاص دارند، عنصر دایره‌ای از نظر هندسی مناسب‌تر است. برخلاف SE مربعی، دیسک در همه‌ی جهت‌ها اثر تقریباً یکسان دارد و جهت‌دار نیست.

چرا عدد ۵ یا ۱۰؟

این عدد شعاع است، یعنی اندازه‌ی عنصر سازنده را تعیین می‌کند.

• شعاع کوچک‌تر `<==` اثر ملایم‌تر

• شعاع بزرگ‌تر `<==` اثر شدیدتر

مثال:

• در **Closing** روی `circles.png`، از شعاع ۱۰ استفاده می‌کنیم چون هدف این است که شکاف‌های بزرگ‌تر بین بخش‌های دایره‌ها پر شود و لبه‌ها هم صاف‌تر شوند.

• در **Opening** روی `blobs.png` و `snowflakes.png`، از شعاع ۵ استفاده می‌کنیم چون می‌خواهیم نقاط و اجزای کوچک‌تر از این اندازه حذف شوند ولی ساختار اصلی حفظ شود. پس عدد شعاع را شانس انتخاب نمی‌کنیم؛ این عدد به مقیاس نویز یا شکاف‌ها در تصویر بستگی دارد.

۱- تمرین اول: Dilation

هدف

گسترش نواحی سفید، پرکردن گپ‌ها و بزرگ‌تر کردن اجسام.

ایده‌ی ریاضی

اگر A تصویر باینری و B عنصر سازنده باشد، آنگاه: $A \oplus B$

دیلاشن یعنی اگر عنصر سازنده با بخشی از شیء برخورد کند، آن قسمت شیء را گسترش می‌دهد.

کد

۱) ساخت تصویر اولیه

در کد ابتدا یک تصویر 256×256 می‌سازیم که $B = \text{zeros}(256, 256)$ یعنی کل تصویر ابتدا سیاه است.

۲) رسم یک مربع سفید

```
for(i=۱۲۸:۱۶۰)
    for(j=۱۲۸:۱۶۰)
        B(i,j)=۱;
    end
end
```

این حلقه یک مربع سفید در وسط تصویر ایجاد می‌کند زیرا می‌خواهیم اثر dilation را روی یک شیء ساده و قابل مشاهده ببینیم.

۳) رسم یک ناحیه‌ی سفید دیگر

```
for(i=۲۰:۲۵)
    for(j=۳۰:۱۹۰)
        B(i,j)=۱;
    end
end
```

این بخش یک نوار سفید باریک اضافه می‌کند تا ببینیم dilation فقط روی یک شکل ساده عمل نمی‌کند، بلکه روی اجسام با فرم متفاوت هم اثر دارد.

۴) اضافه کردن پیکسل‌های تصادفی

```
While(i<۴۰)
    x=uint8(rand*۲۵۵);
    y=uint8(rand*۲۵۵);
    B(x,y)=۱;
    i=i+۱;
end
```

این قسمت ۴۰ پیکسل تصادفی سفید اضافه می‌کند زیرا می‌خواهیم نشان دهیم dilation چگونه **نویزهای سفید** یا نقاط پراکنده را هم بزرگ می‌کند.

در Python/Colab این بخش با random seed قابل تکرار می‌شود، اما چون تصادفی است، اجرای چندباره نتیجه‌های متفاوت می‌دهد؛ به همین دلیل باید حداقل ۳ بار اجرا شود(مطابق خواسته‌ی مسئله).

(۵) چرا kernel های ۳، ۵، ۷، ۹؟

$K_3 = \text{ones}(3);$

$K_5 = \text{ones}(5);$

$K_7 = \text{ones}(7);$

$K_9 = \text{ones}(9);$

این‌ها چهار structuring element مربعی هستند و تمامی عناصرشان ۱ است، یعنی کل ناحیه فعال است.

چرا از **ones** استفاده شده است؟

چون در این مثال هدف فقط دیدن اثر اندازه‌ی **SE** است، نه شکل خاص آن. **SE** مربعی و پر، ساده‌ترین انتخاب برای مشاهده‌ی تفاوت ابعاد است.

نتیجه‌ی این تغییر اندازه چیست؟

• $3 \times 3 \rightarrow$ اثر کم

• $5 \times 5 \rightarrow$ اثر بیشتر

• $7 \times 7 \rightarrow$ بزرگ‌سازی شدیدتر

• $9 \times 9 \rightarrow$ بیشترین گسترش

هرچه **SE** بزرگ‌تر باشد، **dilation** قوی‌تر است.

(۶) **dilation** روی تصویر

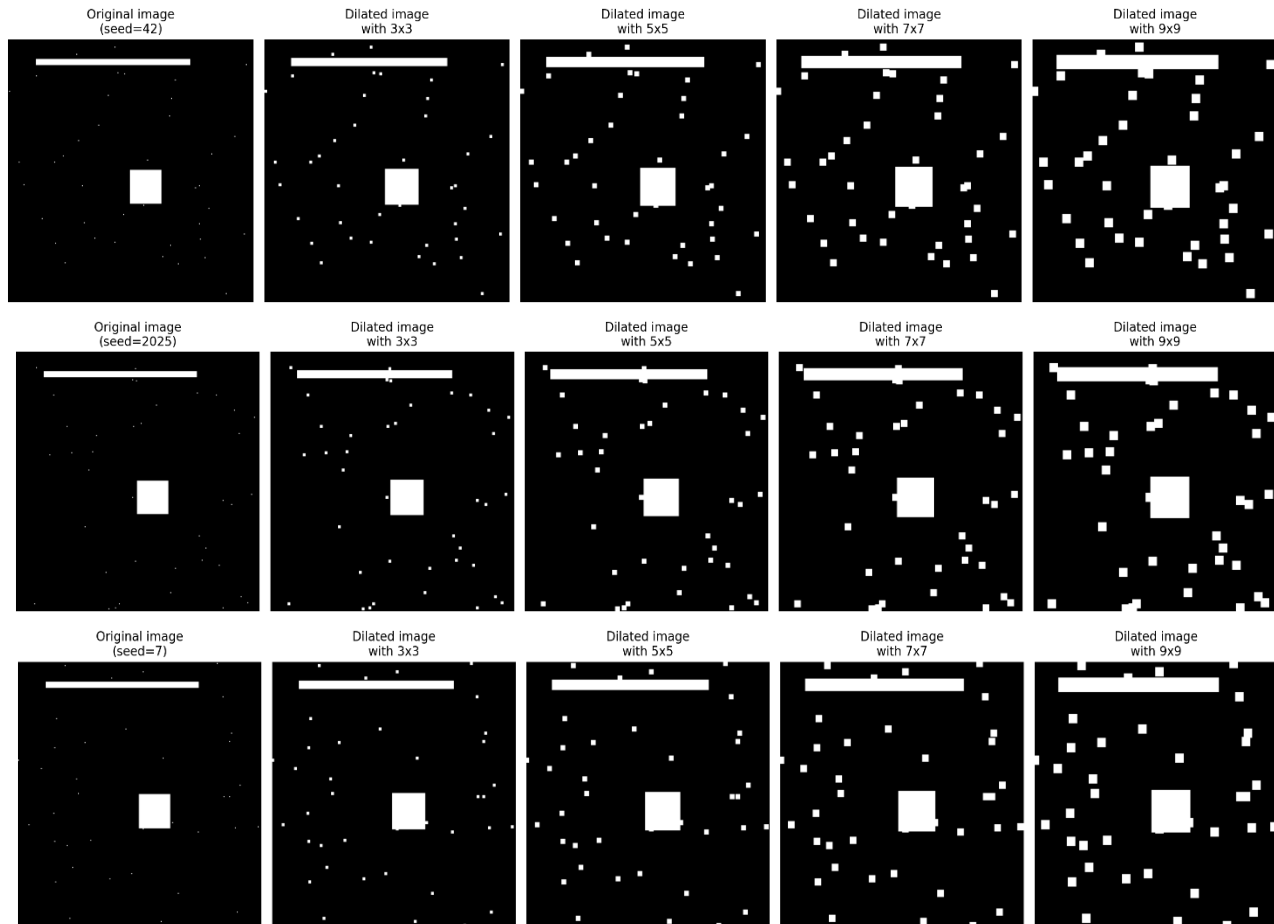
$B_3 = \text{imdilate}(B, K_3);$

$B_5 = \text{imdilate}(B, K_5);$

$B_7 = \text{imdilate}(B, K_7);$

$B_9 = \text{imdilate}(B, K_9);$

در اینجا همان تصویر اولیه با چهار SE مختلف dilate می‌شود.



معنای عملی:

- اجسام سفید بزرگ‌تر می‌شوند.
- فاصله‌ی بین اجسام کوچک می‌شود.
- نقاط پراکنده هم بزرگ‌تر و واضح‌تر می‌شوند.
- لبه‌ها ضخیم‌تر می‌شوند.

کاربرد dilation چیست؟

- پر کردن فاصله‌های کوچک
- وصل کردن اجسام نزدیک به هم
- تقویت نواحی روشن در تصویر باینری
- پیش‌پردازش برای تشخیص شکل‌ها

۲- تمرین دوم: Erosion

هدف

کوچک کردن اجسام سفید، حذف اجسام کوچک و جدا کردن اتصال‌های باریک.

ایده‌ی ریاضی

$A \ominus B$ یعنی فقط آن قسمت‌هایی از شیء باقی می‌مانند که SE به‌طور کامل داخل آن‌ها جا شود.

کد

(۱) خواندن تصویر

```
I = imread('cameraman.tif');
```

در نسخه‌ی پایتون، این تصویر از منبع عمومی بارگذاری می‌شود.

چرا **cameraman**؟ چون یکی از تصویرهای کلاسیک در پردازش تصویر است و برای نمایش مفاهیم به‌خوبی شناخته شده‌است.

۲) تبدیل به باینری

$B = \text{im2bw}(I, 0.3);$

این خط شدت خاکستری را با آستانه‌ی ۰.۳ به باینری تبدیل می‌کند.

چرا این کار لازم است؟ چون عملیات مورفولوژیک کلاسیک روی تصویر باینری تعریف می‌شوند و ما می‌خواهیم تصویر به دو بخش foreground/background تبدیل شود تا erosion روی ساختارها اعمال شود.

۳) ایجاد SE های مختلف

$K_3 = \text{ones}(3);$

$K_5 = \text{ones}(5);$

$K_7 = \text{ones}(7);$

$K_9 = \text{ones}(9);$

این بار هم اندازه‌ی SE مهم است، چون erosion با SE بزرگ‌تر، بخش بیشتری از شیء را حذف می‌کند.

۴) اعمال erosion

$B_3 = \text{imerode}(B, K_3);$

$B_5 = \text{imerode}(B, K_5);$

$B_7 = \text{imerode}(B, K_7);$

$B_9 = \text{imerode}(B, K_9);$

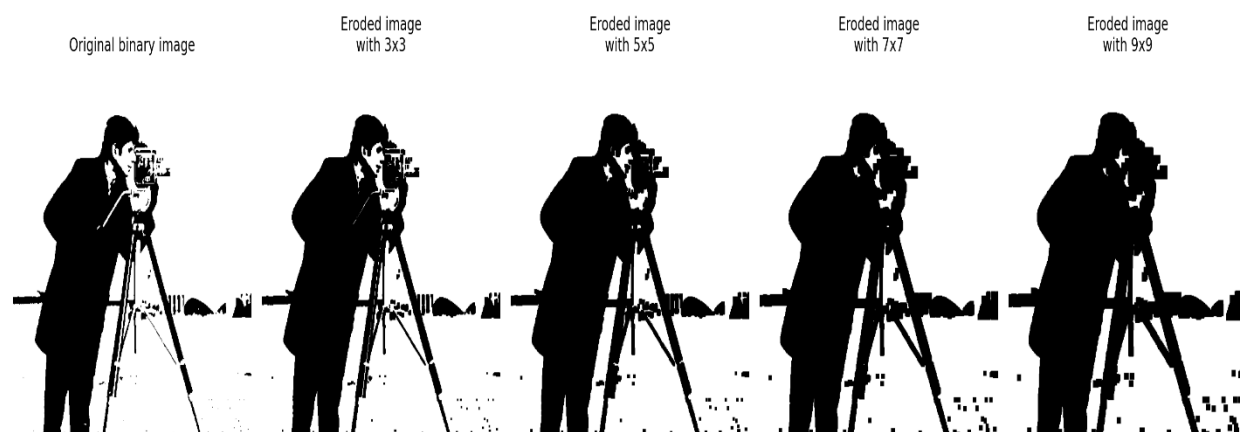
اثر erosion :

- شیء سفید کوچک تر می شود.
- لبه ها عقب می روند.
- بخش های باریک ممکن است حذف شوند.
- اجسام خیلی کوچک ممکن است کاملاً ناپدید شوند.

چرا erosion مفید است؟

- حذف نویزهای سفید کوچک
- جدا کردن اجسام چسبیده
- نازک کردن اشیا

- آماده سازی برای استخراج ویژگی



۳- تمرین سوم: Opening

هدف

حذف اجسام کوچک، حذف برجستگی‌های باریک و صاف کردن شکل اصلی.

ایده‌ی ریاضی

$$A \circ B = (A \ominus B) \oplus B$$

یعنی:

۱. اول erosion

۲. بعد dilation

چرا اول erosion؟ چون erosion اجسام کوچک و قسمت‌های باریک را حذف می‌کند.

چرا بعد dilation؟ چون می‌خواهیم بعد از حذف نویز، اندازه‌ی کلی شیء تا حدی بازسازی شود.

پس opening برای:

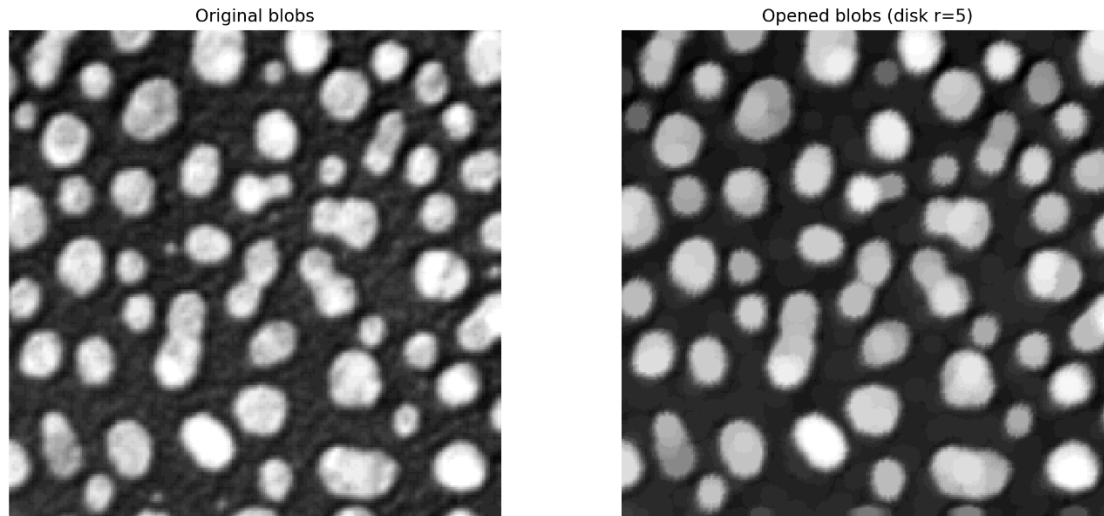
- حذف اشیای کوچک
- حذف noise
- جدا کردن بخش‌های باریک
- نگه داشتن بخش‌های اصلی بسیار مناسب است .

مثال اول : blobs.png

```
I=imread('blobs.png');  
se=strel('disk',5);  
I_opened = imopen(I,se);
```

چرا blobs؟

چون شامل لکه‌ها یا قطعاتی است که بعضی از آن‌ها از بقیه کوچک‌ترند.
opening این لکه‌های کوچک را حذف می‌کند.



چرا disk(5)؟

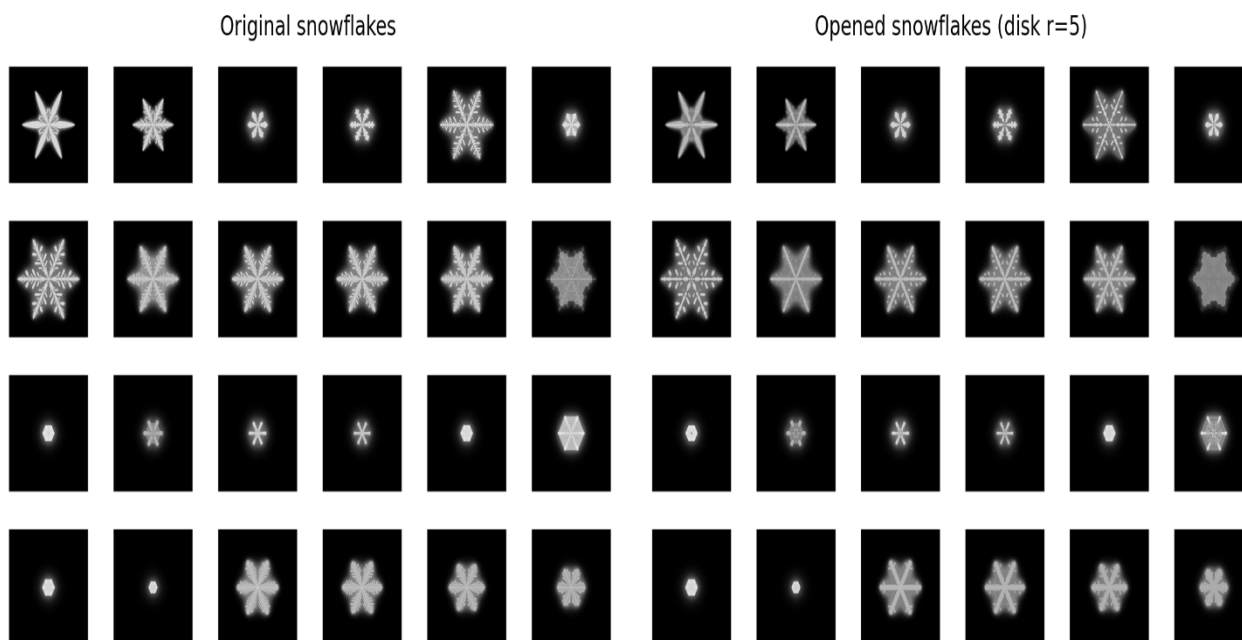
شعاع ۵ یعنی فقط اجزایی که از این مقیاس کوچک‌ترند حذف می‌شوند.
این اندازه تعادل خوبی بین حذف نویز و حفظ ساختار ایجاد می‌کند.

مثال دوم: snowflakes.png

```
I=imread('snowflakes.png');  
se=strel('disk',5);  
I_opened = imopen(I,se);
```

چرا snowflakes؟

چون این تصویر معمولاً شامل اشکال ریز و شاخه‌ای است. Opening اجزای خیلی ریز، زوائد باریک و نویزهای کوچک را حذف می‌کند.



نتیجه:

- اجزای اصلی باقی می‌مانند.
- قسمت‌های نازک یا جداافتاده حذف می‌شوند.
- تصویر تمیزتر و ساده‌تر می‌شود.

۴- تمرین چهارم: Closing

هدف

پر کردن شکاف‌های کوچک، وصل کردن بخش‌های نزدیک و صاف کردن مرزها.

ایده‌ی ریاضی

$$A \cdot B = (A \oplus B) \ominus B$$

یعنی:

۱. اول dilation

۲. بعد erosion

چرا اول dilation و بعد erosion ؟ چون:

- Dilation شکاف‌ها را می‌بندد و بخش‌ها را نزدیک‌تر می‌کند.
- Erosion بعداً شکل را به اندازه‌ی اولیه نزدیک می‌کند اما شکاف‌های بسته‌شده باقی می‌مانند.

پس closing برای:

- بستن gap های کوچک
- پر کردن حفره‌های کوچک
- صاف کردن لبه‌ها

کد

(۱) خواندن تصویر circles

```
originalBW = imread('circles.png');
```


چرا circles؟ چون برای closing عالی است: دایره‌ها معمولاً شکاف‌های کوچک یا ناصافی‌هایی دارند که closing آن‌ها را بهتر نشان می‌دهد.

(۲) ساخت SE دیسکی

`se = strel('disk', 10);`

چرا disk؟ چون دایره‌ها شیء اصلی هستند و SE دیسکی از نظر شکلی مناسب‌تر از مربع است.

چرا 10 = radius؟

چون می‌خواهیم gap‌هایی با اندازه‌ی مشخص بسته شوند. اگر شعاع خیلی کوچک باشد، شکاف‌ها بسته نمی‌شوند و اگر خیلی بزرگ باشد، شکل بیش از حد تغییر می‌کند.

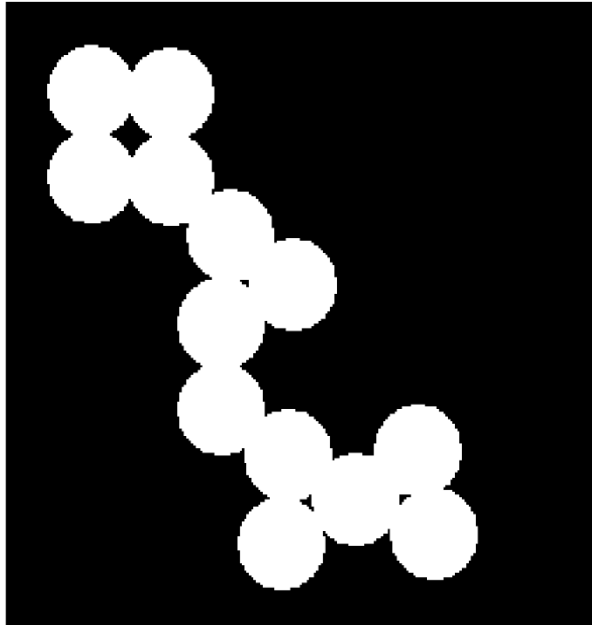
(۳) اعمال closing

`closeBW = imclose(originalBW, se);`

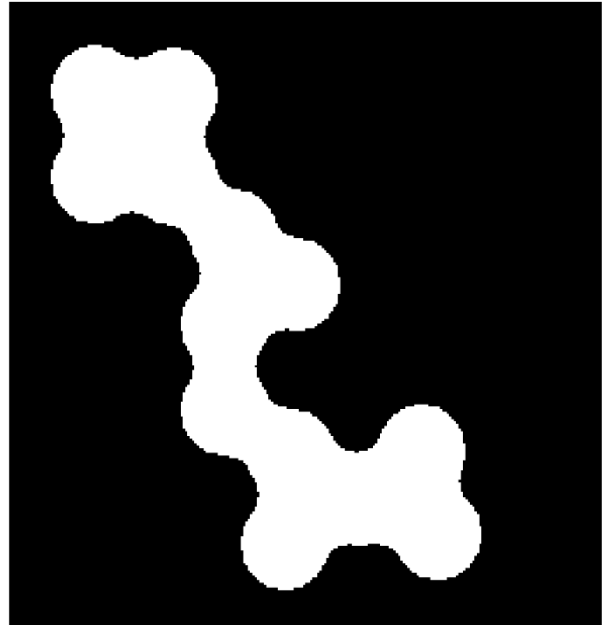
نتیجه:

- فاصله‌های کوچک بین اجزای دایره‌ها پر می‌شود.
- مرزها نرم‌تر و یکنواخت‌تر می‌شوند.
- اجسام به هم نزدیک‌تر و یکپارچه‌تر دیده می‌شوند.

Original circles image



Closed image with disk $r=10$



کاربرد closing چیست؟

- پر کردن حفره‌های کوچک
- بستن شکاف‌ها
- یکپارچه‌سازی قطعات شکسته
- آماده‌سازی برای segmentation

۵- تمرین پنجم: Hit-or-Miss Transform

هدف

تشخیص یک الگوی باینری خاص در تصویر.

ایده‌ی اصلی

Hit-or-Miss برای pattern matching در تصاویر باینری استفاده می‌شود. این عملیات می‌پرسد: "آیا در این نقطه، دقیقاً این شکل خاص وجود دارد یا نه؟"

کد ورودی

```
bw=[000000001100011110011110001100001000];
```

این یک تصویر باینری کوچک مصنوعی است تا بتوان الگو را دقیق بررسی کرد. چرا این ماتریس؟ چون در مثال آموزشی، تصویر کوچک بهتر از تصویر بزرگ است؛ راحت‌تر می‌توان الگو را دید و خروجی را تحلیل کرد.

ماتریس interval یعنی چه؟

```
interval=[0 -1 -1 1 1 -1 0 1 0];
```

در hit-or-miss :

- 1 یعنی این خانه باید حتماً ۱ باشد.
- -1 یعنی این خانه باید حتماً ۰ باشد.
- 0 یعنی فرقی ندارد.

این ماتریس در واقع یک قالب تشخیص الگو است.

چرا این فرمت مهم است؟

چون hit-or-miss فقط وجود pixelهای روشن را نمی‌سنجد، بلکه همزمان وضعیت پس‌زمینه اطراف آن را هم بررسی می‌کند. به همین دلیل برای تشخیص گوشه‌ها، انتهاها، و الگوهای کوچک بسیار مفید است.

خروجی چه چیزی است؟

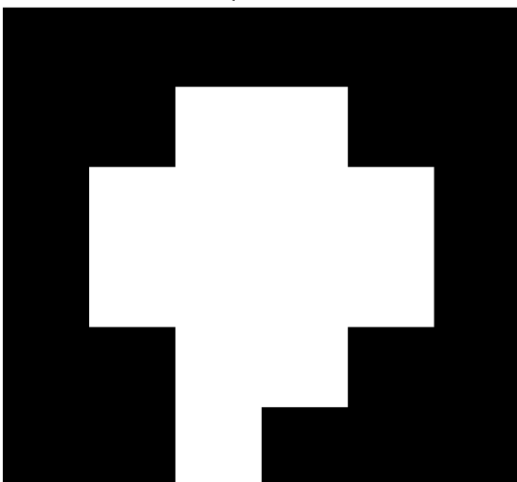
`bw2 = bwhitmiss(bw, interval)`

خروجی، مکان‌هایی را روشن و نشان می‌دهد که الگوی مورد نظر دقیقاً پیدا شده است.

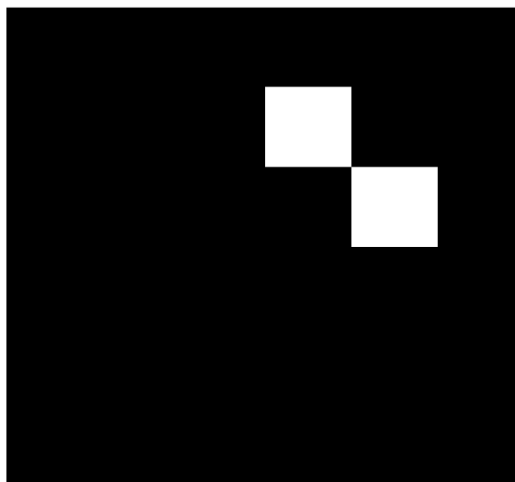
کاربردها:

- تشخیص شکل‌های خاص
- پیدا کردن endpoints در skeletonization
- تشخیص cornerها و اتصال‌ها
- pattern recognition در تصویر باینری

Input bw



bwhitmiss result



جمع‌بندی مفهومی نتایج

Dilation

- اجسام بزرگ‌تر شدند.
- نقاط پراکنده تقویت شدند.
- برای وصل کردن بخش‌های نزدیک مفید است.

Erosion

- اجسام کوچک‌تر شدند.
- بخش‌های باریک حذف شدند.
- برای حذف نویز و جدا کردن اشیا مفید است.

Opening

- اجسام کوچک حذف شدند.
- شکل‌های اصلی حفظ شدند.
- برای تمیز کردن تصویر و حذف نویز عالی است.

Closing

- شکاف‌ها و حفره‌های کوچک بسته شدند و لبه‌ها هموار شدند.
- برای یکپارچه‌سازی اشکال مناسب است.

Hit-or-Miss

- الگوی خاص در تصویر شناسایی شد.
- برای تشخیص شکل‌های دقیق و محلی مفید است.

- [1] <https://github.com>
- [2] <https://stackoverflow.com/questions>
- [3] <https://colab.research.google.com/>
- [4] Textbook - DIP Gonzalez 4th Edition